

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА №2

Атаки на RSA

по дисциплине
«Криптографические методы защиты информации»

Выполнил
студент гр. 5131001/20502

Улановский Г. А

Преподаватель
ассистент ВШК ИКНК

Иванов М. С.

1 ЦЕЛЬ РАБОТЫ

Изучить уязвимости RSA, связанные с неправильным выбором показателей и повторным использованием параметров, реализовать демонстрационные атаки на RSA и сформировать проверки безопасной генерации параметров.

2 ЗАДАЧИ РАБОТЫ

В ходе выполнения лабораторной работы необходимо реализовать набор программ для демонстрации типовых уязвимостей RSA: атаку общего модуля, атаку Винера, ширококестельную атаку при малом общем открытом показателе, бесключевое дешифрование при малом порядке e , а также генерацию и анализ безопасных параметров RSA.

- реализовать восстановление закрытого показателя пользователя при общем модуле RSA и известном закрытом показателе другого пользователя;
- реализовать атаку Винера на RSA с малым закрытым показателем d ;
- реализовать ширококестельную атаку Хостада при малом общем e и отсутствии padding;
- реализовать бесключевое дешифрование при малом порядке e ;
- реализовать генерацию безопасных параметров RSA и проверку устойчивости к реализованным учебным атакам;
- подготовить входные данные, восстановленные значения и журнал успешных проверок.

3 ХОД РАБОТЫ

В ходе работы была разработана программа на языке Python, состоящая из независимых модулей теории чисел, базовых RSA-примитивов, реализаций атак и генератора отчётных данных. CLI-интерфейс позволяет запускать каждую демонстрацию отдельно и формировать JSON-файлы для защиты. Общая структура разработанной программы представлена на рисунке 1.

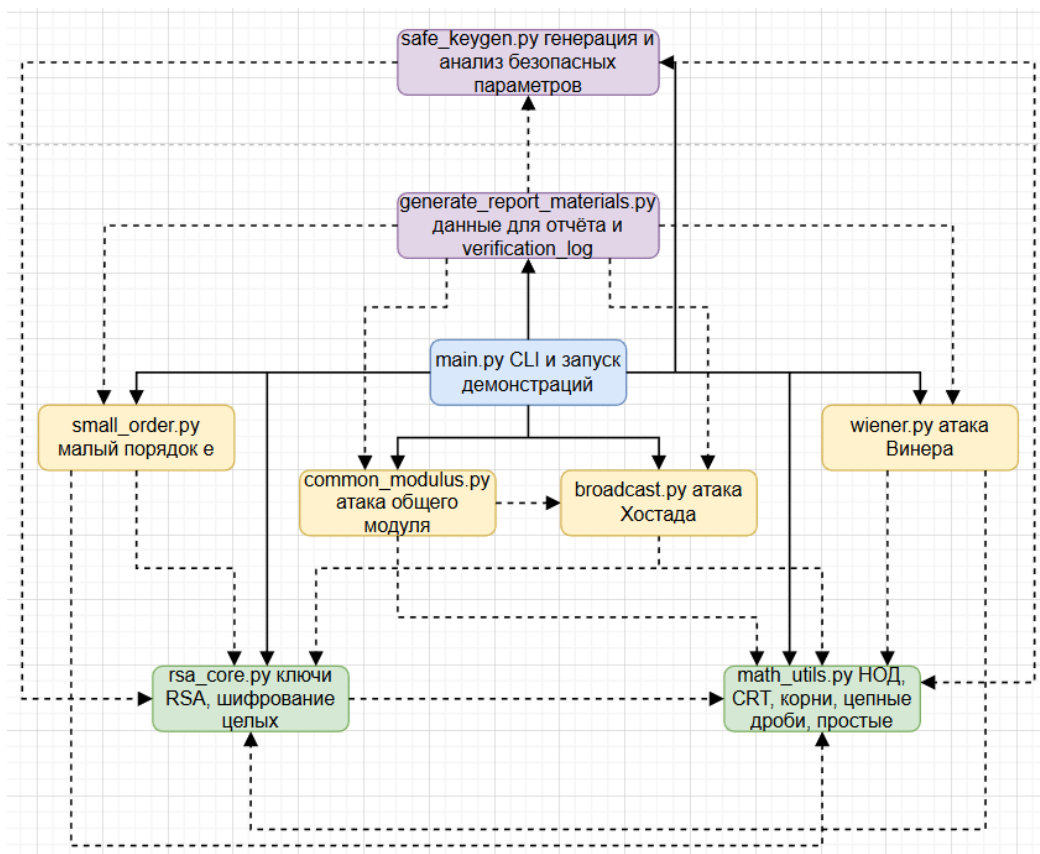


Рисунок 1 – Структура разработанной программы Lab2

3.1 Теоретические сведения

3.1.1 RSA и роль показателей e и d

В RSA выбираются два простых числа p и q , после чего вычисляется модуль n :

$$n = p \cdot q. \quad (1)$$

Функция Эйлера для такого модуля вычисляется по формуле:

$$\varphi(n) = (p - 1)(q - 1). \quad (2)$$

Открытый показатель e выбирается взаимно простым с $\varphi(n)$, а закрытый показатель d является обратным к e по модулю $\varphi(n)$:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}. \quad (3)$$

Шифрование и расшифрование целого представителя сообщения выполняются по формулам:

$$c \equiv m^e \pmod{n}, \quad (4)$$

$$m \equiv c^d \pmod{n}. \quad (5)$$

Безопасность основана на сложности факторизации n ; если p и q восстановлены, то $\varphi(n)$ и d вычисляются напрямую.

3.1.2 Атака общего модуля

Уязвимость возникает, когда разные пользователи используют один и тот же модуль n , но разные пары показателей. Если известны e_b и d_b , то $k = e_b \cdot d_b - 1$ кратно $\varphi(n)$. Представление $k = 2^s \cdot r$, где r нечётно, позволяет искать нетривиальный квадратный корень из единицы по модулю n . Такой корень даёт делители через $\gcd(x - 1, n)$ и $\gcd(x + 1, n)$. После факторизации общего n закрытый показатель другого пользователя находится из условия $e_a \cdot d_a \equiv 1 \pmod{\varphi(n)}$.

3.1.3 Атака Винера

Атака Винера применима, когда закрытый показатель слишком мал: $d < \frac{n^{0.25}}{3}$, а простые множители p и q сравнимы по размеру. В этом случае дробь $\frac{e}{\varphi(n)}$ хорошо приближается дробью $\frac{k}{d}$, а $\varphi(n)$ близко к n . Реализация перебирает подходящие дроби цепной дроби $\frac{e}{n}$, проверяет делимость $e \cdot d - 1$ на k , восстанавливает кандидат $\varphi(n)$ и решает квадратное уравнение для p и q .

3.1.4 Широковещательная атака Хостада при малом e

Малый открытый показатель e не опасен сам по себе, если используется корректный padding. Уязвимость появляется, когда одно и то же сообщение без

padding отправляется нескольким получателям с одинаковым малым e и разными взаимно простыми модулями. Для каждого получателя известно $c_i \equiv m^e \bmod n_i$. Китайская теорема об остатках восстанавливает $x \equiv c_i \bmod n_i$, то есть $x = m^e$, если $m^e < n_1 \cdot n_2 \cdot \dots \cdot n_e$. После этого сообщение находится как точный целочисленный корень степени e .

3.1.5 Бесключевое дешифрование при малом порядке e

Если показатель e имеет малый порядок в мультипликативной группе по модулю $\varphi(n)$, повторное применение операции шифрования быстро образует цикл. Для шифртекста $c_0 = c$ вычисляются значения $c_i = c_{i-1}^e \bmod n$. Когда очередное значение снова равно c_0 , предыдущее значение является исходным сообщением. Уязвимость связана не с факторизацией n , а с плохими алгебраическими свойствами выбранного e .

3.1.6 Безопасная генерация параметров RSA

Для учебной безопасной генерации требуется выбирать разные простые p и q близкой длины, проверять $\gcd(e, \varphi(n)) = 1$, использовать стандартный открытый показатель $e = 65537$ и избегать малого d . Дополнительная проверка $d > \frac{n^{0.25}}{3}$ исключает демонстрационный случай атаки Винера. В практических системах также обязателен padding, например OAEP для шифрования и PSS для подписи.

3.2 Описание реализации

Программа разделена на модули: math_utils.py содержит функции теории чисел, rsa_core.py реализует базовые RSA-примитивы, common_modulus.py реализует атаку общего модуля, wiener.py — атаку Винера, broadcast.py — широковебательную атаку, small_order.py — бесключевое дешифрование при малом порядке e , safe_keygen.py — генерацию и анализ безопасных параметров.

Скрипт `scripts/generate_report_materials.py` формирует данные для отчёта и `verification log`.

3.3 Атака общего модуля RSA

В демонстрации два пользователя имеют общий модуль n , но разные открытые показатели e_a и e_b . По известной паре e_b, d_b программа восстановила множители p, q , а затем вычислила закрытый показатель d_a для второго пользователя. Схема атаки общего модуля представлена на рисунке 2.

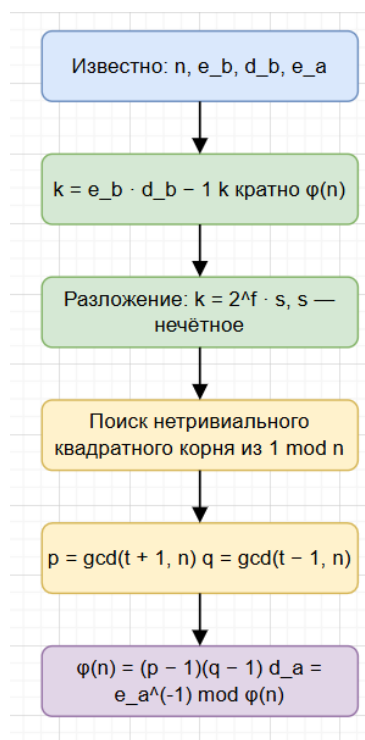


Рисунок 2 – Атака общего модуля RSA

Параметры и результат атаки общего модуля были сформированы скриптом генерации отчётных данных. В демонстрации использовались общий модуль n , открытые показатели e_a и e_b , а также известный закрытый показатель d_b одного пользователя. В результате программа восстановила множители p и q общего модуля, после чего вычислила закрытый показатель d_a другого пользователя.

Проверка результата:

$p \cdot q = n$ — выполнено;

$e_a \cdot d_a \equiv 1 \bmod \varphi(n)$ — выполнено.

Для ручной проверки результата атаки общего модуля были подготовлены контрольные значения, приведённые в листинге 1. В них указаны восстановленные множители p , q , общий модуль n , открытый показатель e_a , восстановленный закрытый показатель d_a , а также тестовое сообщение $m = 12345$.

Листинг 1 – Контрольные значения для проверки атаки общего модуля RSA

```
=====
=====
1. COMMON MODULUS ATTACK - check recovered d_a
=====
=====
p = 227345361425643384226787680439708178353
q = 233757747727612912297308686944562987339
n                                     =
53143739643178526046505830386125140694662700970495598769397328180
499492872667
phi      =      (p      -      1)      *      (q      -      1)      =
53143739643178526046505830386125140694201597861342342472873231813
115221706976
e_a = 17
d_a                                     =
21882716323661746019149459570757410874083010884082141018241918981
870973644049
e_b = 65537
d_b                                     =
29902628180750588621556510083140068789224073771073747974879431410
355019541121

p                                     *                                     q                                     =
53143739643178526046505830386125140694662700970495598769397328180
499492872667
(e_a * d_a) mod phi = 1
(e_b * d_b) mod phi = 1
test message m = 12345
c      =      m^e_a      mod      n      =
35921015229160754168041499167232261877904445075120405270301818847
65625
m' = c^d_a mod n = 12345
```


Полученные значения были проверены в CrypTool. Тестовое сообщение шифровалось с использованием открытого показателя e_a , после чего расшифровывалось восстановленным закрытым показателем d_a . Результат проверки приведён на рисунке 3.

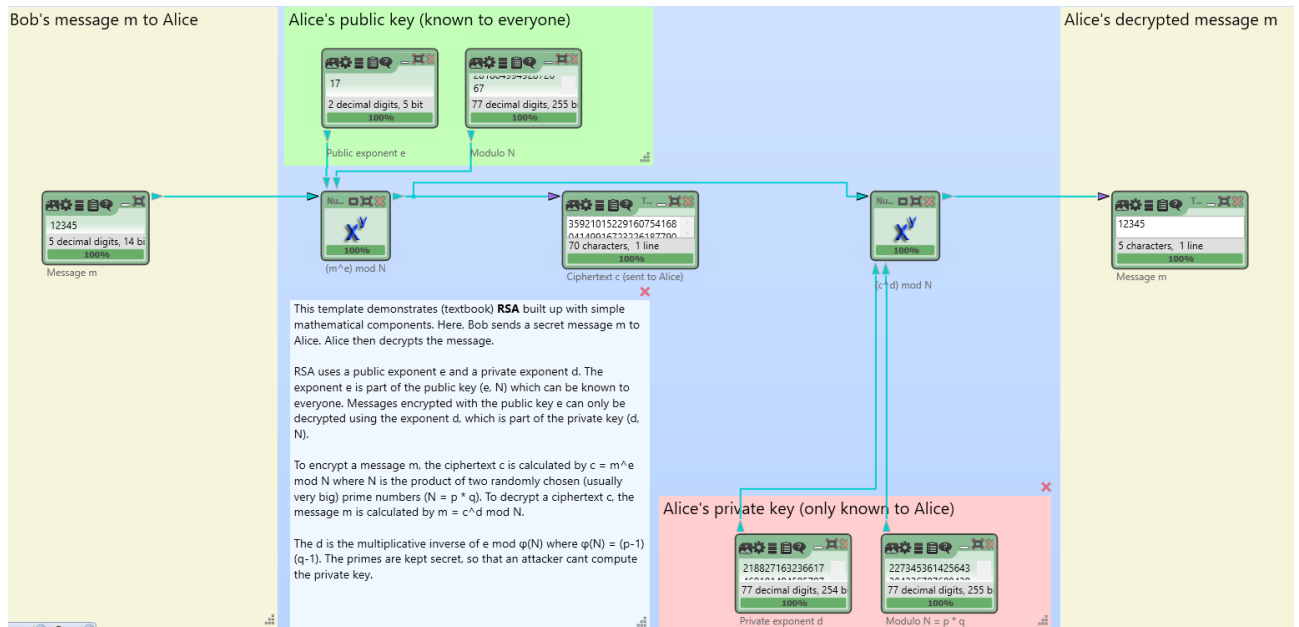


Рисунок 3 – Проверка восстановленного закрытого показателя d_a в CrypTool

3.4 Атака Винера

Для демонстрации был сгенерирован учебный ключ RSA с малым закрытым показателем $d < \frac{n^{0.25}}{3}$. Атака Винера использует подходящие дроби цепной дроби $\frac{e}{n}$ и восстанавливает d . После этого по восстановленному $\phi(n)$ находятся p и q . Последовательность атаки представлена на рисунке 4.

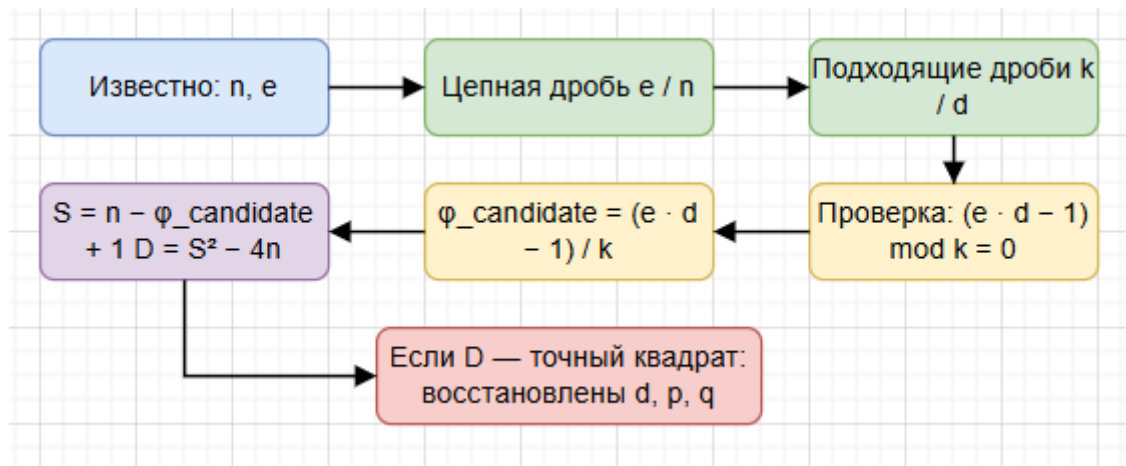


Рисунок 4 – Атака Винера на RSA с малым d

Для атаки Винера был сгенерирован учебный RSA-ключ с заведомо малым закрытым показателем d . Программа по открытому ключу (n, e) восстановила значение d , а затем восстановила множители p и q .

Проверка результата:

$$d_{real} = recovered_d;$$

$$d_{real} < \frac{n^{0.25}}{3};$$

$$p \cdot q = n.$$

Для проверки результата атаки Винера были подготовлены значения, приведённые в листинге 2. В них указаны параметры открытого ключа, восстановленный закрытый показатель и проверка того, что $recovered_d$ находится ниже границы атаки Винера.

Листинг 2 – Контрольные значения для проверки атаки Винера

```

2. WIENER ATTACK - check recovered d
p = 295562450443724194010148863598810025321
q = 300808040366286013671830620673267643853
n
88907561523834196864882340192320433825753196354926486711579433022
424940001813
phi
88907561523834196864882340192320433825156825864116476503897453538
152862332640
e
69694828551511179123355382286377816080381173145646381801779306503
599603042269
d_real = 5217288787482220309
  
```

```

recovered_d = 5217288787482220309
wiener_bound = 5755902384281297644
p * q =
88907561523834196864882340192320433825753196354926486711579433022
424940001813
(e * recovered_d) mod phi = 1
recovered_d < wiener_bound = True
test message m = 12345
c = m^e mod n =
52379860554629160665917013793526296828459765312220473625385751228
619693542624
m' = c^d mod n = 12345

```

Для найденного атакой Винера закрытого показателя d была выполнена ручная проверка в CrypTool. Проверялось, что значение d корректно обращает RSA-преобразование и восстанавливает тестовое сообщение. Результат проверки приведён на рисунке 5.

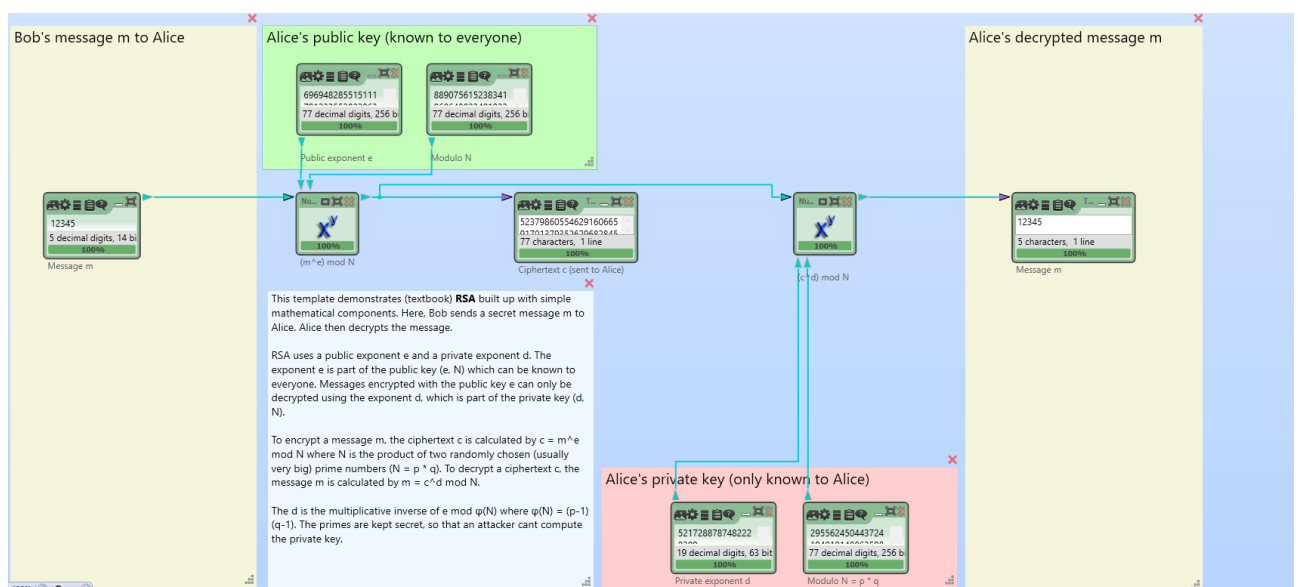


Рисунок 5 – Проверка результата атаки Винера в CrypTool

Из рисунка 5 видно, что восстановленный атакой Винера закрытый показатель d корректно расшифровывает тестовый шифртекст. Это подтверждает успешное восстановление закрытого ключа для уязвимого RSA-параметра.

3.5 Широковещательная атака при малом общем e

Одно сообщение было зашифровано для трёх получателей с одинаковым $e = 3$ и попарно взаимно простыми модулями. Китайская теорема об остатках восстанавливает целое значение m^e , после чего берётся точный целочисленный корень. Схема широковещательной атаки представлена на рисунке 6.

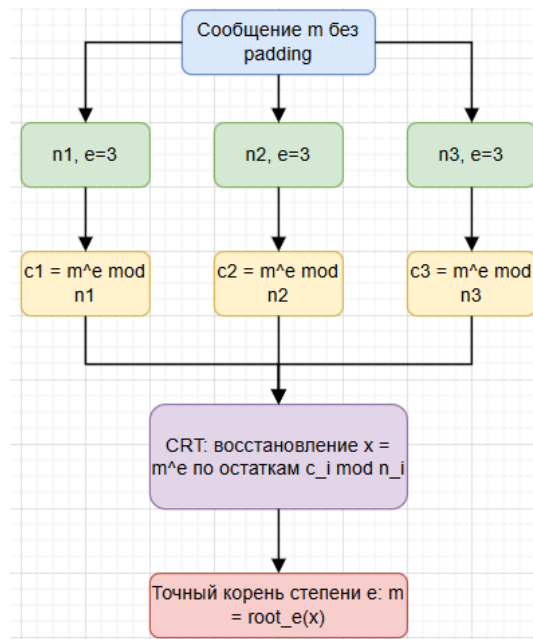


Рисунок 6 – Широковещательная атака Хостада

В широковещательной атаке одно и то же сообщение было зашифровано для трёх получателей с одинаковым открытым показателем $e = 3$ и разными попарно взаимно простыми модулями n_1, n_2, n_3 . По значениям c_1, c_2, c_3 с помощью китайской теоремы об остатках было восстановлено m^e , после чего извлечением точного корня степени e восстановлено исходное сообщение.

Проверка результата:

$$recovered_{message} = message_{int}.$$

Контрольные значения для широковещательной атаки приведены в листинге 3. Для каждого получателя указаны модуль n_i , шифртекст c_i и проверка равенства $c_i = m^e \bmod n_i$ при общем $e = 3$.

Листинг 3 – Контрольные значения для проверки широковещательной атаки

```
=====
=====
3. BROADCAST ATTACK - check  $c_i = m^e \bmod n_i$ 
=====
=====
e = 3
message_int = 1389079653420110012978
recovered_message = 1389079653420110012978
message_text = KMZI Lab2

Recipient 1:
n_1 = 123328053040051166654605450399278176737
c_1 = 23023426393225951804560781732378546971
 $m^e \bmod n_1 = 23023426393225951804560781732378546971$ 
check = True

Recipient 2:
n_2 = 108888719020153424457020321423659652443
c_2 = 9734800150069383221132727140630881760
 $m^e \bmod n_2 = 9734800150069383221132727140630881760$ 
check = True

Recipient 3:
n_3 = 144339737591145761349161739110028180691
c_3 = 23435269099579804197765798921848827221
 $m^e \bmod n_3 = 23435269099579804197765798921848827221$ 
check = True

Pairwise gcd of moduli:
gcd(n_1, n_2) = 1
gcd(n_1, n_3) = 1
gcd(n_2, n_3) = 1
```

В CrypTool была выполнена проверка исходных данных широковещательной атаки. Для получателя вычислялось значение $c_i = m^e \bmod n_i$ и сравнивалось с шифртекстом, сформированным программой. Результат проверки приведён на рисунке 7.

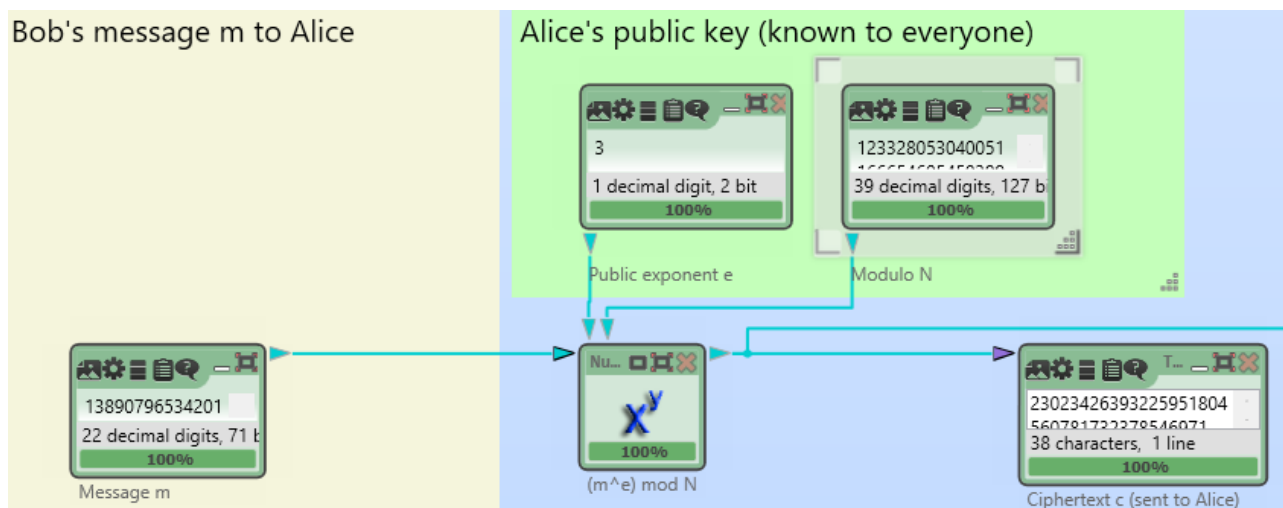


Рисунок 7 – Проверка исходных данных широковещательной атаки в CrypTool

Проверка подтвердила, что шифртексты c_1 , c_2 и c_3 соответствуют одному и тому же сообщению, зашифрованному с одинаковым открытым показателем $e = 3$ и разными взаимно простыми модулями. Это подтверждает выполнение условий широковещательной атаки Хостада.

3.6 Бесключевое дешифрование при малом порядке e

Для учебного малого RSA-модуля показатель e имеет малый порядок по модулю $\phi(n)$. Повторное возведение шифртекста в степень e возвращает цикл, где предыдущий элемент является исходным сообщением. Демонстрация показывает, что небезопасный выбор порядка e может позволить расшифровать сообщение без знания закрытого ключа. Схема атаки представлена на рисунке 8.

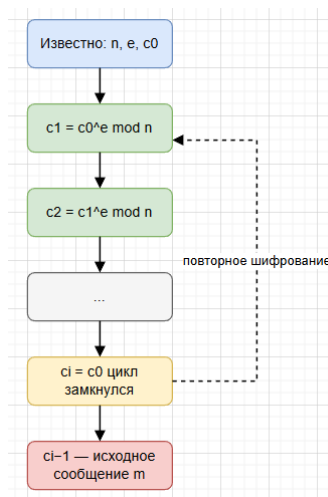


Рисунок 8 – Бесключевое дешифрование при малом порядке e

Для демонстрации бесключевого дешифрования при малом порядке e использовался малый учебный RSA-модуль. Повторное возведение шифртекста в степень e образовало короткий цикл. Предыдущий элемент перед возвратом к исходному шифртексту оказался исходным сообщением.

Проверка результата:

$$m = 123;$$

$$recovered_m = 123;$$

$$iterations = 2.$$

Для проверки атаки при малом порядке e были использованы небольшие учебные параметры RSA. Контрольные значения приведены в листинге 4.

Листинг 4 – Контрольные значения для проверки атаки при малом порядке e

```

=====
=====
4. SMALL ORDER ATTACK - check short cycle
=====
=====
p = 23
q = 47
n = 1081
phi = 1012
e = 45
d = 45
m = 123
c = 859
recovered_m = 123
  
```

```
order = 2
```

```
Check in CrypTool:
```

```
p * q = 1081
```

```
(e * d) mod phi = 1
```

```
e^order mod phi = 1
```

```
m^e mod n = 859
```

```
c^e mod n = 123
```

```
(c^e mod n)^e mod n = 859
```

В CrypTool была проверена короткая циклическая последовательность повторного RSA-преобразования. Сообщение $m = 123$ после шифрования даёт $c = 859$, а повторное возведение c в степень e по модулю n возвращает исходное сообщение. Результат проверки приведён на рисунке 9.

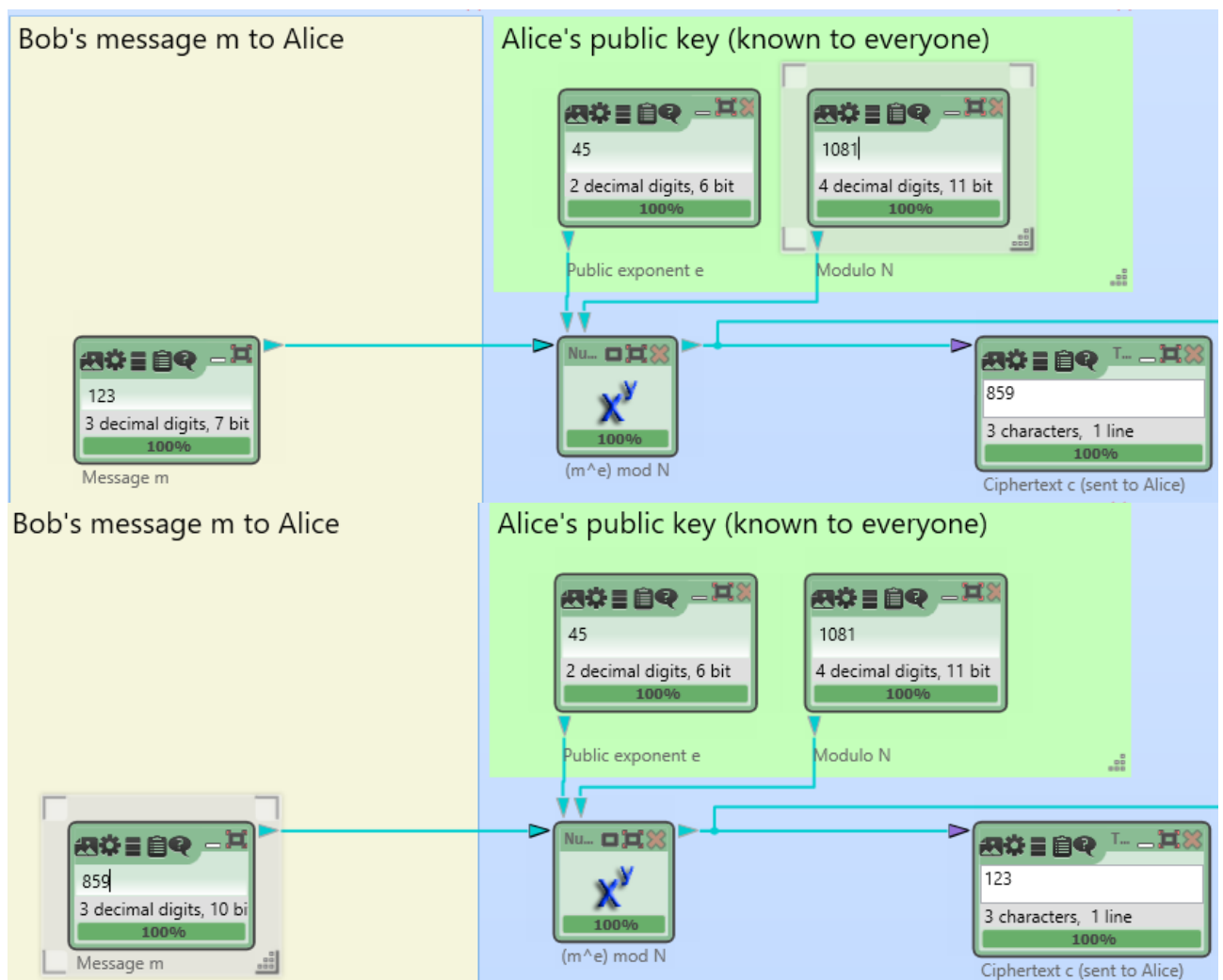


Рисунок 9 – Проверка цикла бесключевого дешифрования в CrypTool

Из рисунка 9 видно, что для выбранных параметров повторное применение RSA-преобразования образует короткий цикл. Поэтому сообщение может быть восстановлено без использования закрытого ключа.

3.7 Генерация и проверка безопасных параметров RSA

Для сравнения с уязвимыми учебными параметрами была реализована генерация более безопасной RSA-пары. В программе проверяются различие p и q , близость их битовых длин, условие $\gcd(e, \varphi(n)) = 1$, использование открытого показателя $e = 65537$, а также выполнение неравенства $d > \frac{n^{0.25}}{3}$. Дополнительно к сгенерированному ключу применяется собственная реализация атаки Винера; для безопасного ключа закрытый показатель не восстанавливается. Порядок генерации и проверки параметров представлен на рисунке 10.

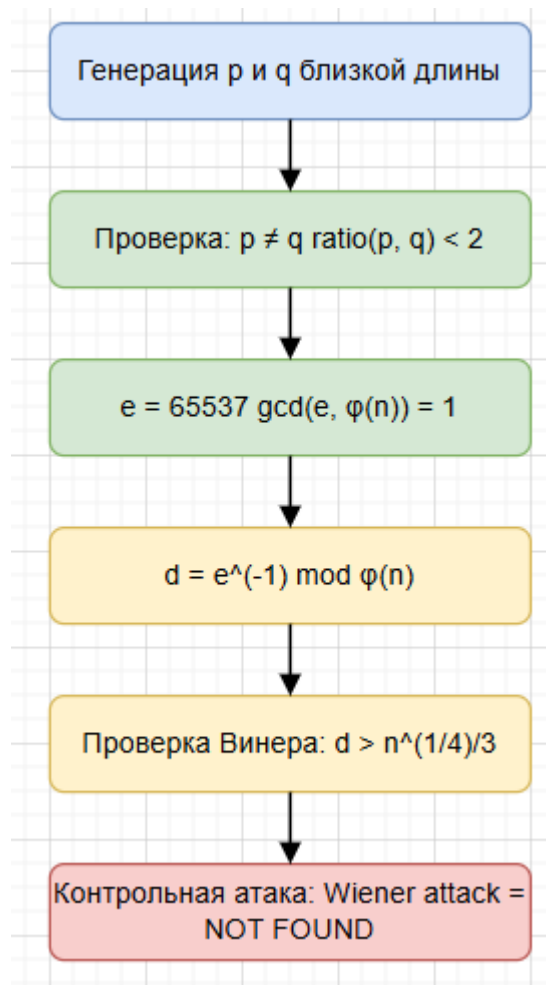


Рисунок 10 – Проверка безопасных параметров RSA

Результат генерации безопасных учебных параметров RSA и их программного анализа приведён на рисунке 11.

```

(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> cd C:\Users\Georgul\Documents\8_sem\KMZI\Lab2
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2>
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> python -m src.main safe-keygen --bits 512 --out artifacts\safe_key.json
{
  "private_key": {
    "n": 6793189418663384135868099826321439886252014261849854953384240064972468577150747217603762495000435197143082157517579903043271480213285
461745062980692538327,
    "e": 65537,
    "d": 3615149722366340365086465929819992659304080159460112474009828657797042686807158352580091832736819440968104135310183826695048877280303
403041620141659900169,
    "p": 101457274451633460205209250520836470248068868718903686228495050818720712865669,
    "q": 66956159185035291615264644385138582569456386514090696301909632608820464666283
  },
  "analysis": {
    "modulus_bits": 512,
    "p_bits": 256,
    "q_bits": 256,
    "gcd_e_phi": 1,
    "d": 3615149722366340365086465929819992659304080159460112474009828657797042686807158352580091832736819440968104135310183826695048877280303
403041620141659900169,
    "wiener_bound": 95696747275657850394778651670809108320,
    "d_gt_wiener_bound": true,
    "p_q_ratio_ok": true,
    "e_value": 65537,
    "cryptool_manual_check": "not_performed",
    "notes": [
      "p and q are distinct probable primes with similar bit length",
      "gcd(e, phi(n)) equals 1",
      "d is above the Wiener vulnerability bound",
      "e defaults to 65537 and is not the small broadcast demo exponent"
    ]
  }
}
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2>

```

Рисунок 11 – Генерация и проверка безопасных параметров RSA

На рисунке 11 показано, что для сгенерированного ключа выполнены условия $modulus_{bits} = 512$, $gcd(e, \varphi(n)) = 1$, $d_gt_wiener_bound = true$ и $p_q_ratio_ok = true$. Это означает, что ключ не подпадает под реализованную демонстрационную атаку Винера и проходит базовые проверки безопасной генерации.

Для сгенерированной безопасной учебной RSA-пары были подготовлены контрольные значения, приведённые в листинге 5. Они позволяют проверить корректность модуля, открытого и закрытого показателей, а также восстановление тестового сообщения.

Листинг 5 – Контрольные значения для проверки безопасной RSA-пары

```
=====
=====
5. SAFE KEYGEN - check valid RSA key pair
=====
=====
p                                                    =
10138125871254716941163258518911345777049936894241321796259777414
9773773992681
q                                                    =
91201492259735210843359402007604554944213353627033411836733419547
449506009511
n                                                    =
92461220817545835779862603296922455068660323005000863811937546876
39021951006925850476144789634625704415135322835526660002654505355
744120326359430030388991
phi                                                    =
92461220817545835779862603296922455068660323005000863811937546876
39021951006733267725172507254370712427938604822811947280085058725
944789132662206750386800
e = 65537
d                                                    =
54930461152191084671085805870968701467847012774458834437306992807
60872784266096400184317127270838208773391970623864125110244774119
881294000033614901907473
wiener_bound = 103363843806430368626711493283703392470

Check in CrypTool:
p                                                    *                               q                               =
92461220817545835779862603296922455068660323005000863811937546876
```

```

39021951006925850476144789634625704415135322835526660002654505355
744120326359430030388991
gcd(e, phi) = 1
(e * d) mod phi = 1
d > wiener_bound = True
p_bits = 256
q_bits = 256
modulus_bits = 512
test message m = 12345
c = m^e mod n =
29299838252697410559933170947662091658366970729303508601309261533
95271755866140863243522488757292138275490037077839923221447909155
293899699137114949698717
m' = c^d mod n = 12345

```

Сгенерированная RSA-пара была дополнительно проверена в CrypTool. Проверялись корректность шифрования и расшифрования тестового сообщения, а также соответствие закрытого показателя выбранному открытому показателю $e = 65537$. Результат проверки приведён на рисунке 12.

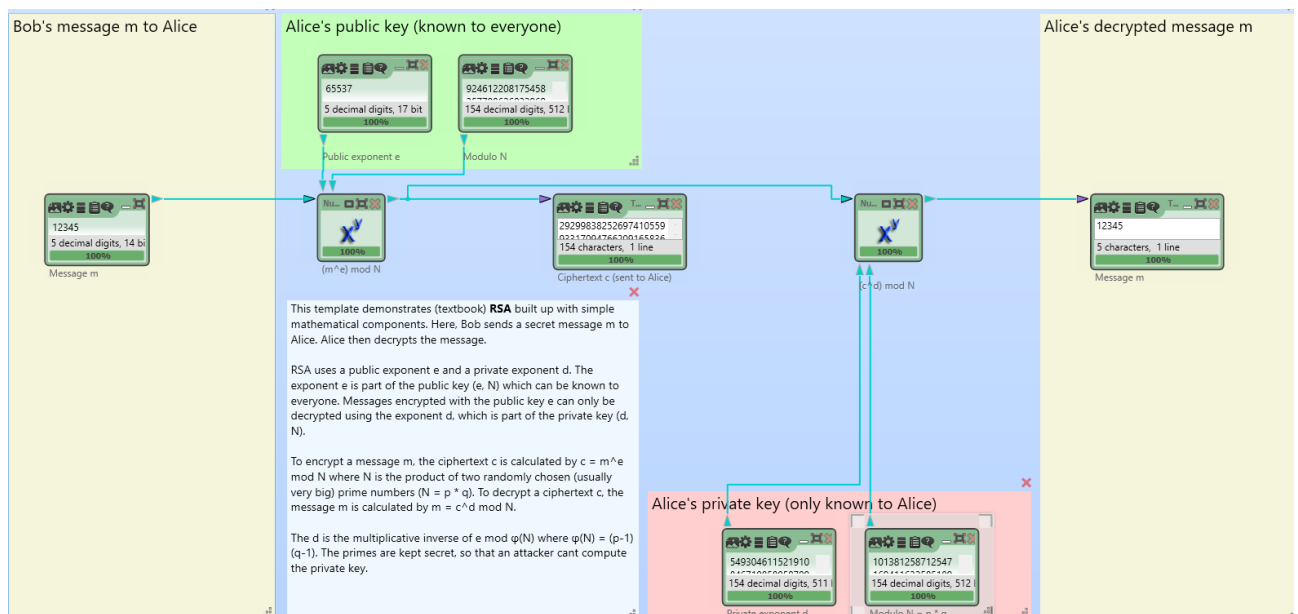


Рисунок 12 – Проверка безопасной RSA-пары в CrypTool

Из рисунка 12 видно, что сгенерированная RSA-пара корректно выполняет шифрование и расшифрование тестового сообщения. Дополнительно

программная проверка подтвердила $\gcd(e, \varphi(n)) = 1, d > \frac{n^{0.25}}{3}$ и отрицательный результат атаки Винера для данного ключа.

3.8 Итоговая проверка

Корректность реализации была подтверждена автоматическими тестами и демонстрационным запуском основных режимов программы. Итоговая схема проверки представлена на рисунке 13.

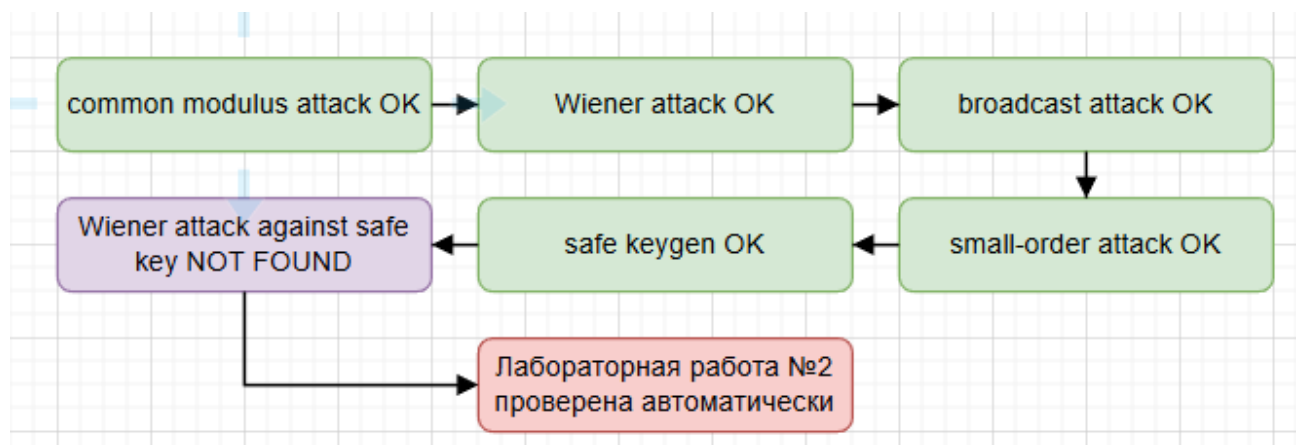


Рисунок 13 – Итоговая схема проверки реализованных атак

Результаты автоматического запуска тестов и формирования журнала проверки приведены на рисунке 14.

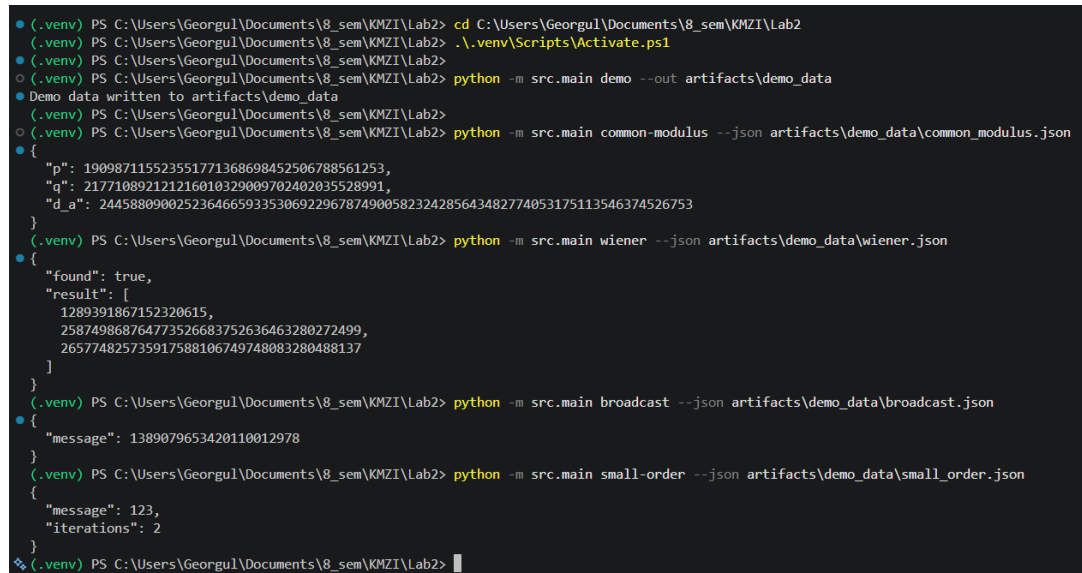
```
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab1> cd C:\Users\Georgul\Documents\8_sem\KMZI\Lab2
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2>
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> pytest -q
..... [100%]
9 passed in 0.34s
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> python scripts\generate_report_materials.py
Generated Lab2 report materials:
- report/generated/report_data.json
- report/generated/report_data.md
- report/generated/verification_log.txt
- report/generated/theory.md
- report/generated/control_questions.md
- report/generated/stand_commands.md
- report/generated/full_report_draft.md
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> Get-Content .\report\generated\verification_log.txt -Encoding UTF8
common modulus attack OK
Wiener attack OK
broadcast attack OK
small-order attack OK
safe keygen OK
Wiener attack against safe key NOT FOUND
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2>
```

Рисунок 14 – Проверка тестов и журнала выполнения ЛР2

Из рисунка 14 видно, что все автоматические тесты завершились успешно, а журнал проверки содержит успешные результаты для атаки общего модуля,

атаки Винера, широковещательной атаки, атаки при малом порядке e и генерации безопасных параметров RSA.

Пример отдельного запуска CLI-команд для демонстрации реализованных атак RSA приведён на рисунке 15.



```
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> cd C:\Users\Georgul\Documents\8_sem\KMZI\Lab2
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> python -m src.main demo --out artifacts\demo_data
Demo data written to artifacts\demo_data
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2>
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> python -m src.main common-modulus --json artifacts\demo_data\common_modulus.json
{
  "p": 190987115523551771368698452506788561253,
  "q": 217710892121216010329009702402035528991,
  "d_a": 24458809002523646659335306922967874900582324285643482774053175113546374526753
}
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> python -m src.main wiener --json artifacts\demo_data\wiener.json
{
  "found": true,
  "result": [
    1289391867152320615,
    258749868764773526683752636463280272499,
    265774825735917588106749748083280488137
  ]
}
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> python -m src.main broadcast --json artifacts\demo_data\broadcast.json
{
  "message": 1389079653420110012978
}
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> python -m src.main small-order --json artifacts\demo_data\small_order.json
{
  "message": 123,
  "iterations": 2
}
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab2> █
```

Рисунок 15 – Демонстрационный запуск CLI-команд для атак RSA

В результате выполнения CLI-команд были получены восстановленные значения для атаки общего модуля, найденный закрытый показатель в атаке Винера, восстановленное сообщение в широковещательной атаке и сообщение $m = 123$ при атаке с малым порядком e .

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 При каком условии возможна атака Винера?

Атака Винера возможна, если закрытый показатель RSA d выбран слишком малым. Для классической формулировки атаки предполагается, что модуль $n = p \cdot q$, множители p и q близки по размеру, например $q < p < 2q$, а закрытый показатель удовлетворяет условию: $d < \frac{n^{0.25}}{3}$.

В этом случае дробь $\frac{k}{d}$, связанная с равенством $e \cdot d = 1 + k \cdot \varphi(n)$, является хорошим приближением к $\frac{e}{n}$. Поэтому значение d может появиться среди знаменателей подходящих дробей разложения $\frac{e}{n}$ в цепную дробь. Перебирая подходящие дроби, можно восстановить d , затем вычислить $\varphi(n)$, а после этого восстановить p и q .

Таким образом, атака Винера применима при сочетании двух условий: закрытый показатель d мал, а простые множители p и q имеют близкий размер.

4.2 Перечислите методы противодействия атаке Винера?

Для противодействия атаке Винера необходимо исключить возможность выбора слишком малого закрытого показателя d . Основные меры защиты:

- генерировать RSA-ключи так, чтобы d было существенно больше границы $\frac{n^{0.25}}{3}$;
- использовать стандартный открытый показатель $e = 65537$ и проверять, что $\gcd(e, \varphi(n)) = 1$;
- выбирать простые p и q близкой битовой длины, но не допускать параметров, приводящих к малому d ;
- после генерации ключа выполнять проверку $d > \frac{n^{0.25}}{3}$;
- не использовать самодельную генерацию RSA-параметров в прикладных системах, а применять проверенные криптографические

библиотеки.

В данной лабораторной работе такая проверка реализована в программе генерации безопасных параметров RSA: ключ принимается только при выполнении условия $d > \frac{n^{0.25}}{3}$, а затем дополнительно проверяется собственной реализацией атаки Винера.

4.3 Почему числа p и q необходимо выбирать так, чтобы $p - 1$, $q - 1$ имели большие простые делители?

Если $p - 1$ или $q - 1$ имеют только малые простые делители, то мультипликативные группы F_p^* и F_q^* имеют “гладкие” порядки. Это может приводить к появлению малых порядков элементов, используемых в RSA, в частности открытого показателя e .

В контексте лабораторной работы это важно из-за атаки бесключевого дешифрования при малом порядке e . Если порядок e по модулю $\varphi(n)$ мал, то повторное применение операции RSA-шифрования быстро образует цикл. В таком случае открытый текст может быть восстановлен без знания закрытого ключа.

Кроме того, если малый порядок возникает хотя бы в одной из групп F_p^* или F_q^* , возможно получение нетривиального делителя n через вычисление НОД разностей элементов последовательности. Поэтому p и q выбирают так, чтобы $p - 1$ и $q - 1$ содержали большие простые делители: это увеличивает порядки соответствующих групп и снижает риск атак, связанных с малыми порядками.

4.4 Предложите способы защиты от атаки бесключевого дешифрования широковещательных сообщений без отказа от использования общих малых закрытых показателей.

В формулировке методички, вероятно, имеется в виду использование общего малого открытого показателя e , поскольку широковещательная атака

выполняется именно при одинаковом малом e у нескольких получателей. Отказываться от малого e полностью не обязательно, но необходимо устранить условия применимости атаки.

Основные способы защиты:

- не шифровать одно и то же сообщение нескольким получателям в виде textbook RSA без дополнения;
- использовать вероятностное дополнение, например RSA-OAEP, чтобы одинаковые сообщения перед RSA-преобразованием превращались в разные псевдослучайные представители;
- добавлять уникальную случайность для каждого получателя;
- использовать гибридное шифрование: сообщение шифровать симметричным алгоритмом, а для каждого получателя отдельно защищать случайный сеансовый ключ;
- проверять, что сообщение после дополнения имеет достаточную длину и не удовлетворяет условию $m^e < n_1 \cdot n_2 \cdot \dots \cdot n_e$ в виде обычного целого числа;
- не применять textbook RSA напрямую в прикладных протоколах.

Таким образом, общий малый открытый показатель e сам по себе не является достаточным условием атаки. Опасность возникает при отсутствии padding и при отправке одного и того же сообщения нескольким получателям. Использование RSA-OAEP или иной стандартизированной схемы дополнения устраняет прямую алгебраическую связь между шифртекстами.

5 ВЫВОДЫ

В ходе лабораторной работы были реализованы и исследованы атаки на криптосистему RSA, использующие некорректный выбор показателей и повторное использование параметров. Были реализованы атака общего модуля, атака Винера, ширококвещательная атака при малом общем открытом показателе e , бесключевое дешифрование при малом порядке e , а также генерация и проверка более безопасных учебных параметров RSA.

В результате атаки общего модуля показано, что использование одного и того же модуля n разными пользователями недопустимо: знание закрытого показателя одного пользователя позволяет восстановить множители p и q , а затем вычислить закрытый показатель другого пользователя. В атаке Винера продемонстрировано восстановление закрытого показателя d при его слишком малом значении. В ширококвещательной атаке показано, что одно и то же сообщение, отправленное нескольким получателям с одинаковым малым e без padding, может быть восстановлено с помощью китайской теоремы об остатках и извлечения целочисленного корня. В атаке при малом порядке e продемонстрировано восстановление сообщения без знания закрытого ключа за счёт короткого цикла повторного возведения в степень.

Также была реализована генерация более безопасных учебных параметров RSA. Для сгенерированного ключа проверялись условия $p \neq q$, близость битовых длин p и q , $\gcd(e, \varphi(n)) = 1$, использование $e = 65537$, выполнение ограничения $d > \frac{n^{0.25}}{3}$ и отрицательный результат атаки Винера против безопасного ключа. Автоматические тесты и демонстрационный журнал подтвердили корректность реализованных атак и проверок.

Таким образом, безопасность RSA зависит не только от сложности факторизации модуля n , но и от корректного выбора параметров, отсутствия общего модуля между пользователями, достаточного размера закрытого показателя d , отсутствия малых порядков e и обязательного применения стандартизованных схем дополнения. В практических системах textbook RSA

использовать нельзя; следует применять проверенные реализации RSA-OAEP и RSA-PSS. Полный исходный код разработанной программы приведён в приложениях 1–8.

ПРИЛОЖЕНИЕ 1. ЛИСТИНГ ФАЙЛА SRC/MATH_UTILS.PY

```
"""Вспомогательные функции теории чисел для учебных атак на RSA."""

from __future__ import annotations

import math
import secrets
from collections.abc import Iterable, Iterator, Sequence

gcd = math.gcd

def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
    """Вернуть gcd(a, b) и коэффициенты Безу x, y."""
    # Расширенный алгоритм Евклида одновременно находит НОД и
    # коэффициенты
    # x, y такие, что  $a \cdot x + b \cdot y = \text{gcd}(a, b)$ . Эти коэффициенты нужны
    # для
    # вычисления обратного элемента по модулю.
    old_r, r = a, b
    old_s, s = 1, 0
    old_t, t = 0, 1
    while r:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
        old_t, t = t, old_t - q * t
    if old_r < 0:
        return -old_r, -old_s, -old_t
    return old_r, old_s, old_t

def mod_inverse(a: int, m: int) -> int:
    """Вернуть обратный элемент к a по модулю m."""
    # Обратный элемент существует только при  $\text{gcd}(a, m) = 1$ . В RSA
    # это условие
    # используется для вычисления закрытого показателя d из e.
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("inverse does not exist")
    return x % m

def integer_nth_root(value: int, n: int) -> tuple[int, bool]:
    """Вернуть floor(value ** (1/n)) и признак точного корня."""
```

```

if value < 0:
    raise ValueError("value must be non-negative")
if n <= 0:
    raise ValueError("n must be positive")
if value in (0, 1) or n == 1:
    return value, True

# Двоичный поиск работает только с целыми числами, поэтому не
возникает
# ошибок округления, которые опасны для больших RSA-параметров.
low, high = 0, 1 << ((value.bit_length() + n - 1) // n)
while low <= high:
    mid = (low + high) // 2
    power = mid**n
    if power == value:
        return mid, True
    if power < value:
        low = mid + 1
    else:
        high = mid - 1
return high, False

def continued_fraction(numerator: int, denominator: int) ->
list[int]:
    """Вернуть простую цепную дробь для numerator / denominator."""
    # Цепные дроби используются в атаке Винера: малый d проявляется
    среди
    # знаменателей подходящих дробей разложения e/n.
    if denominator == 0:
        raise ZeroDivisionError("denominator must be non-zero")
    terms: list[int] = []
    while denominator:
        q = numerator // denominator
        terms.append(q)
        numerator, denominator = denominator, numerator - q *
denominator
    return terms

def convergents(cf: Sequence[int]) -> Iterator[tuple[int, int]]:
    """Порождать числитель и знаменатель подходящих дробей."""
    # Подходящие дроби строятся рекуррентно и дают лучшие
    рациональные
    # приближения исходной дроби на каждом шаге.
    p_nm2, p_nm1 = 0, 1

```

```

q_nm2, q_nm1 = 1, 0
for a in cf:
    p = a * p_nm1 + p_nm2
    q = a * q_nm1 + q_nm2
    yield p, q
    p_nm2, p_nm1 = p_nm1, p
    q_nm2, q_nm1 = q_nm1, q

def crt(residues: Sequence[int], moduli: Sequence[int]) -> int:
    """Решить систему  $x \equiv \text{residue}_i \pmod{\text{modulus}_i}$  для взаимно
    простых модулей."""
    # Китайская теорема об остатках объединяет несколько сравнений
    в одно.
    # В широковежательной атаке она восстанавливает целое значение
     $m^e$ .
    if len(residues) != len(moduli) or not residues:
        raise ValueError("residues and moduli must have equal non-
        zero length")
    total_modulus = 1
    for modulus in moduli:
        if modulus <= 1:
            raise ValueError("moduli must be greater than one")
        total_modulus *= modulus

    result = 0
    for residue, modulus in zip(residues, moduli):
        partial = total_modulus // modulus
        result += residue * partial * mod_inverse(partial %
        modulus, modulus)
    return result % total_modulus

def is_probable_prime(n: int, rounds: int = 32) -> bool:
    """Вероятностный тест Миллера-Рабина с предварительной
    проверкой малых простых."""
    if n < 2:
        return False
    small_primes = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37)
    for prime in small_primes:
        if n == prime:
            return True
        if n % prime == 0:
            return False

```

```

    # Представляем n-1 как 2^r * d, где d нечетно. Это стандартная
    форма для
    # раундов Миллера-Рабина.
    d = n - 1
    r = 0
    while d % 2 == 0:
        r += 1
        d //= 2

    # Тест вероятностный: несколько независимых оснований резко
    снижают шанс
    # принять составное число за простое. Для лабораторной этого
    достаточно.
    for _ in range(rounds):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_prime(bits: int) -> int:
    """Сгенерировать вероятно простое число ровно bits бит."""
    if bits < 2:
        raise ValueError("bits must be at least 2")
    # Кандидат принудительно делается нечетным и с установленным
    старшим битом,
    # чтобы получить число нужной длины.
    while True:
        candidate = secrets.randbits(bits) | (1 << (bits - 1)) | 1
        if is_probable_prime(candidate):
            return candidate

```

ПРИЛОЖЕНИЕ 2. ЛИСТИНГ ФАЙЛА SRC/RSA_CORE.PY

```
"""Минимальные RSA-примитивы для учебной лабораторной Lab2."""

from __future__ import annotations

import json
from dataclasses import asdict, dataclass
from pathlib import Path

from .math_utils import gcd, generate_prime, mod_inverse

@dataclass(frozen=True)
class PublicKey:
    # Открытый ключ содержит только модуль n и открытый показатель e.
    n: int
    e: int

@dataclass(frozen=True)
class PrivateKey:
    # Закрытый ключ дополнительно хранит d и множители p, q. В реальных системах p и q защищают особенно строго; здесь они нужны для проверок.
    n: int
    e: int
    d: int
    p: int
    q: int

def generate_keypair_from_primes(p: int, q: int, e: int) -> tuple[PrivateKey, PublicKey]:
    if p == q:
        raise ValueError("p and q must be different")
    # Для RSA с  $n = p \cdot q$  функция Эйлера равна  $(p-1) \cdot (q-1)$ .
    phi = (p - 1) * (q - 1)
    # Открытый показатель должен быть взаимно прост с  $\phi(n)$ , иначе обратный
    # показатель d не существует.
    if gcd(e, phi) != 1:
        raise ValueError("e must be coprime with phi")
    n = p * q
```



```

    # Замкнутый показатель d определяется сравнением  $e \cdot d \equiv 1 \pmod{\phi(n)}$ .
    d = mod_inverse(e, phi)
    private = PrivateKey(n=n, e=e, d=d, p=p, q=q)
    public = PublicKey(n=n, e=e)
    return private, public

def generate_keypair(bits: int = 1024, e: int = 65537) ->
tuple[PrivateKey, PublicKey]:
    if bits < 16:
        raise ValueError("bits must be at least 16")
    half = bits // 2
    # Генерация повторяется, пока случайные простые не дадут
допустимый e.
    while True:
        p = generate_prime(bits - half)
        q = generate_prime(half)
        if p == q:
            continue
        try:
            return generate_keypair_from_primes(p, q, e)
        except ValueError:
            continue

def encrypt_int(m: int, public_key: PublicKey) -> int:
    # Учебное RSA-шифрование одного целого представителя:  $c = m^e \pmod{n}$ .
    # Padding здесь не реализован, потому что Lab2 изучает
математические атаки.
    if not 0 <= m < public_key.n:
        raise ValueError("message representative out of range")
    return pow(m, public_key.e, public_key.n)

def decrypt_int(c: int, private_key: PrivateKey) -> int:
    # Обратная операция:  $m = c^d \pmod{n}$ .
    if not 0 <= c < private_key.n:
        raise ValueError("ciphertext representative out of range")
    return pow(c, private_key.d, private_key.n)

def save_public_key(path: str | Path, public_key: PublicKey) ->
None:

```

```

    # JSON используется только как простой формат обмена для
    демонстраций.
    Path(path).write_text(json.dumps(asdict(public_key),
    indent=2), encoding="utf-8")

def save_private_key(path: str | Path, private_key: PrivateKey) -
> None:
    # Такие файлы нельзя коммитить как реальные секреты; artifacts
    игнорируется.
    Path(path).write_text(json.dumps(asdict(private_key),
    indent=2), encoding="utf-8")

def load_public_key(path: str | Path) -> PublicKey:
    # При чтении JSON явно приводим значения к int, чтобы формат
    был устойчив
    # к строковым и числовым представлениям больших чисел.
    data = json.loads(Path(path).read_text(encoding="utf-8"))
    return PublicKey(n=int(data["n"]), e=int(data["e"]))

def load_private_key(path: str | Path) -> PrivateKey:
    # Загрузка закрытого ключа нужна для CLI-демонстраций и
    локальных проверок.
    data = json.loads(Path(path).read_text(encoding="utf-8"))
    return PrivateKey(
        n=int(data["n"]),
        e=int(data["e"]),
        d=int(data["d"]),
        p=int(data["p"]),
        q=int(data["q"]),
    )

```

ПРИЛОЖЕНИЕ 3. ЛИСТИНГ ФАЙЛА SRC/COMMON_MODULUS.PY

```
"""Восстановление закрытого показателя RSA при общем модуле."""

from __future__ import annotations

import secrets

from .math_utils import gcd, mod_inverse

def recover_from_known_private_exponent(
    n: int,
    e_b: int,
    d_b: int,
    e_a: int,
    max_attempts: int = 128,
) -> tuple[int, int, int]:
    """Восстановить p, q и d_a по n, e_b, d_b и другому e_a."""
    # Так как  $e_b \cdot d_b \equiv 1 \pmod{\phi(n)}$ , число  $k = e_b \cdot d_b - 1$  кратно  $\phi(n)$ .
    # Это позволяет использовать k как замену неизвестного порядка группы.
    k = e_b * d_b - 1
    if k <= 0:
        raise ValueError("invalid exponent data")

    # Выделяем степень двойки:  $k = 2^f \cdot s$ , где s нечетно. Далее возведение
    # случайных оснований в степень s помогает найти квадратный корень из 1.
    f = 0
    s = k
    while s % 2 == 0:
        f += 1
        s //= 2
    if f == 0:
        raise ValueError("e_b*d_b - 1 must be even")

    # Ищем нетривиальный квадратный корень из единицы по модулю n.
    Для RSA
    # такой корень раскрывает факторизацию n.
    for _ in range(max_attempts):
        a = secrets.randbelow(n - 3) + 2
        b = pow(a, s, n)
        if b in (1, n - 1):
```

```

        continue
    t = b
    for _ in range(f):
        next_t = pow(t, 2, n)
        if next_t == 1 and t not in (1, n - 1):
            # Если  $t^2 \equiv 1 \pmod{n}$ , но  $t$  не равен  $\pm 1$ , то  $\gcd(t-$ 
1, n) и
            #  $\gcd(t+1, n)$  дают разные простые множители  $p$  и  $q$ .
            p = gcd(t + 1, n)
            q = gcd(t - 1, n)
            if p in (1, n) or q in (1, n):
                break
            if p * q != n:
                p, q = q, p
            if p * q != n:
                break
            phi = (p - 1) * (q - 1)
            # После факторизации общего  $n$  закрытый показатель
для  $e_a$ 
            # вычисляется обычным обратным элементом по модулю
phi(n).
            d_a = mod_inverse(e_a, phi)
            if (e_a * d_a) % phi != 1:
                raise ValueError("failed to verify recovered
d_a")
            return min(p, q), max(p, q), d_a
    t = next_t
    if t == n - 1:
        break

    raise ValueError("failed to factor n within max_attempts")

```

ПРИЛОЖЕНИЕ 4. ЛИСТИНГ ФАЙЛА SRC/WIENER.PY

```
"""Атака Винера на RSA с малым закрытым показателем."""

from __future__ import annotations

import secrets

from .math_utils import continued_fraction, convergents, gcd,
generate_prime, integer_nth_root, mod_inverse
from .rsa_core import PrivateKey, PublicKey

def wiener_attack(n: int, e: int) -> tuple[int, int, int] | None:
    """Вернуть d, p, q, если атака Винера применима, иначе None."""
    # Атака перебирает подходящие дроби цепной дроби e/n. При малом
    d
    # настоящая дробь k/d появляется среди этих приближений.
    for k, d in convergents(continued_fraction(e, n)):
        if k == 0:
            continue
        ed_minus_1 = e * d - 1
        if ed_minus_1 % k != 0:
            continue
        # Из  $e*d - 1 = k*\phi(n)$  получаем кандидата на  $\phi(n)$ .
        phi_candidate = ed_minus_1 // k
        s = n - phi_candidate + 1
        discriminant = s * s - 4 * n
        if discriminant < 0:
            continue
        # p и q являются корнями уравнения  $x^2 - (p+q)x + n = 0$ .
        Поэтому
        # дискриминант должен быть точным квадратом.
        root, exact = integer_nth_root(discriminant, 2)
        if not exact or (s + root) % 2 != 0:
            continue
        p = (s + root) // 2
        q = (s - root) // 2
        if p > 1 and q > 1 and p * q == n:
            return d, min(p, q), max(p, q)
    return None

def generate_wiener_vulnerable_key(bits: int = 512) ->
tuple[PrivateKey, PublicKey]:
    """Сгенерировать учебный RSA-ключ с  $d < n^{(1/4)}/3$ ."""
```

```
if bits < 64:
    raise ValueError("bits must be at least 64")
half = bits // 2
# Ключ специально делается уязвимым для отчета и тестов. Это
не безопасная
# генерация, а контролируемый пример применения атаки Винера.
while True:
    q = generate_prime(half)
    p = generate_prime(bits - half)
    if p == q:
        continue
    if p < q:
        p, q = q, p
    if not q < p < 2 * q:
        continue
    n = p * q
    phi = (p - 1) * (q - 1)
    bound, _ = integer_nth_root(n, 4)
    # Условие Винера: d должен быть меньше примерно  $n^{1/4}/3$ .
    max_d = max(3, bound // 3)
    for _ in range(256):
        d = secrets.randbelow(max_d - 2) + 2
        if gcd(d, phi) != 1:
            continue
        e = mod_inverse(d, phi)
        if e >= n:
            continue
        private = PrivateKey(n=n, e=e, d=d, p=p, q=q)
        public = PublicKey(n=n, e=e)
        if wiener_attack(n, e) is not None:
            return private, public
```

ПРИЛОЖЕНИЕ 5. ЛИСТИНГ ФАЙЛА SRC/BROADCAST.PY

```
"""Широковещательная атака Хостада для малого общего RSA-
показателя."""

from __future__ import annotations

from collections.abc import Sequence

from .math_utils import crt, gcd, integer_nth_root
from .rsa_core import PublicKey, encrypt_int, generate_keypair

def recover_broadcast_message(e: int, pairs: Sequence[tuple[int,
int]]) -> int:
    # Атака требует как минимум e получателей, которым отправлено
одно и то же
    # сообщение без padding с одинаковым малым открытым показателем
e.
    if len(pairs) < e:
        raise ValueError("at least e ciphertexts are required")
    moduli = [n for n, _ in pairs]
    ciphertexts = [c for _, c in pairs]
    # Модули должны быть попарно взаимно простыми, иначе CRT в этой
форме не
    # применима и общий делитель уже сам по себе раскрывает
факторизацию.
    for i, n_i in enumerate(moduli):
        for n_j in moduli[i + 1 :]:
            if gcd(n_i, n_j) != 1:
                raise ValueError("moduli must be pairwise
coprime")
    # CRT восстанавливает x, сравнимый с m^e по каждому модулю n_i.
При
    # условии m^e < product(n_i) это не просто остаток, а само
целое m^e.
    x = crt(ciphertexts, moduli)
    # Последний шаг - точное извлечение корня степени e. Если
корень не точный,
    # входные данные не соответствуют классической атаке Хостада.
    root, exact = integer_nth_root(x, e)
    if not exact:
        raise ValueError("combined ciphertext is not an exact e-
th power")
    return root
```

```

def generate_broadcast_demo(
    e: int = 3,
    recipients: int = 3,
    bits: int = 256,
    message_int: int | None = None,
) -> dict[str, object]:
    # Генератор создает учебный набор получателей для отчета: одно
    # сообщение,
    # одинаковый e и разные взаимно простые RSA-модули.
    if recipients < e:
        raise ValueError("recipients must be at least e")
    message = message_int if message_int is not None else
int.from_bytes(b"KMZI Lab2", "big")
    public_keys: list[PublicKey] = []
    pairs: list[tuple[int, int]] = []
    product = 1
    seen_moduli: set[int] = set()
    while len(pairs) < recipients:
        private, public = generate_keypair(bits, e)
        if public.n in seen_moduli or message >= public.n:
            continue
        seen_moduli.add(public.n)
        public_keys.append(public)
        pairs.append((public.n, encrypt_int(message, public)))
        product *= public.n
    if message**e >= product:
        raise ValueError("message^e must be smaller than product
of moduli")
    return {
        "e": e,
        "message_int": message,
        "pairs": pairs,
        "public_keys": public_keys,
        "recovered_message": recover_broadcast_message(e, pairs),
    }

```


ПРИЛОЖЕНИЕ 6. ЛИСТИНГ ФАЙЛА SRC/SMALL_ORDER.PY

```
"""Демонстрация атаки при малом порядке открытого показателя."""

from __future__ import annotations

from .math_utils import mod_inverse
from .rsa_core import PublicKey, encrypt_int


def recover_small_order_message(
    n: int,
    e: int,
    c: int,
    max_iterations: int = 100_000,
) -> tuple[int, int]:
    """Восстановить m, когда повторное возведение в степень
    возвращает шифртекст."""
    # Если e имеет малый порядок по модулю phi(n), повторное
    применение
    # операции RSA-шифрования образует короткий цикл.
    c0 = c
    previous = c0
    current = c0
    for iteration in range(1, max_iterations + 1):
        current = pow(previous, e, n)
        if current == c0:
            # Когда цикл вернулся к исходному шифртексту c0,
            предыдущий элемент
            # цикла является открытым текстом для учебного примера.
            return previous, iteration
        previous = current
    raise ValueError("cycle was not found within max_iterations")


def generate_small_order_demo() -> dict[str, int]:
    """Вернуть детерминированный малый учебный RSA-пример."""
    # Малые p и q выбраны только для наглядной лабораторной
    демонстрации.
    # Такие параметры не имеют промышленной криптографической
    стойкости.
    p = 23
    q = 47
    n = p * q
    phi = (p - 1) * (q - 1)
```

```
# Для выбранного phi показатель e имеет малый порядок, поэтому
цикл
# повторного шифрования быстро раскрывает сообщение.
e = 45
d = mod_inverse(e, phi)
order = 2
assert pow(e, order, phi) == 1
message = 123
public = PublicKey(n=n, e=e)
c = encrypt_int(message, public)
recovered, iterations = recover_small_order_message(n, e, c)
return {
    "p": p,
    "q": q,
    "n": n,
    "phi": phi,
    "e": e,
    "d": d,
    "m": message,
    "c": c,
    "recovered_m": recovered,
    "iterations": iterations,
    "order": order,
}
```

ПРИЛОЖЕНИЕ 7. ЛИСТИНГ ФАЙЛА SRC/SAFE_KEYGEN.PY

```
"""Учебная генерация и проверка более безопасных параметров RSA
для Lab2."""

from __future__ import annotations

from .math_utils import gcd, generate_prime, integer_nth_root
from .rsa_core import PrivateKey, generate_keypair_from_primes

def generate_safe_rsa_params(bits: int = 1024, e: int = 65537) ->
PrivateKey:
    # В лабораторной безопасный вариант использует
    # распространенный показатель
    # e = 65537 и проверяет базовые условия, но это не промышленный
    # генератор.
    if bits < 64:
        raise ValueError("bits must be at least 64")
    if e < 17:
        raise ValueError("use a larger public exponent for safe
parameters")
    half = bits // 2
    # p и q генерируются разными и близкими по длине. Это снижает
    # риск простых
    # атак, связанных с несбалансированными множителями.
    while True:
        p = generate_prime(bits - half)
        q = generate_prime(half)
        if p == q:
            continue
        try:
            private, _ = generate_keypair_from_primes(p, q, e)
        except ValueError:
            # Здесь отбрасываются случаи gcd(e, phi(n)) != 1.
            continue
        analysis = analyze_rsa_params(private)
        # Ключ принимается только если d выше границы Винера, а p
        # и q имеют
        # допустимое соотношение размеров.
        if analysis["d_gt_wiener_bound"] and
analysis["p_q_ratio_ok"]:
            return private
```

```

def analyze_rsa_params(private_key: PrivateKey) -> dict[str,
object]:
    # Отчетная проверка параметров: она фиксирует размер модуля,
    взаимную
    # простоту e и phi(n), границу Винера и соотношение p/q.
    p, q = sorted((private_key.p, private_key.q))
    phi = (p - 1) * (q - 1)
    fourth_root, _ = integer_nth_root(private_key.n, 4)
    wiener_bound = fourth_root // 3
    ratio = q / p
    return {
        "modulus_bits": private_key.n.bit_length(),
        "p_bits": private_key.p.bit_length(),
        "q_bits": private_key.q.bit_length(),
        "gcd_e_phi": gcd(private_key.e, phi),
        "d": private_key.d,
        "wiener_bound": wiener_bound,
        "d_gt_wiener_bound": private_key.d > wiener_bound,
        "p_q_ratio_ok": ratio < 2,
        "e_value": private_key.e,
        "cryptool_manual_check": "not_performed",
        "notes": [
            "p and q are distinct probable primes with similar bit
length",
            "gcd(e, phi(n)) equals 1",
            "d is above the Wiener vulnerability bound",
            "e defaults to 65537 and is not the small broadcast
demo exponent",
        ],
    }

```

ПРИЛОЖЕНИЕ 8. ЛИСТИНГ ФАЙЛА SRC/MAIN.PY

```
"""CLI для запуска учебных демонстраций KMZI Lab2."""

from __future__ import annotations

import argparse
import json
import shutil
from dataclasses import asdict
from pathlib import Path
from typing import Any

from .broadcast import generate_broadcast_demo,
recover_broadcast_message
from .common_modulus import recover_from_known_private_exponent
from .rsa_core import generate_keypair, save_private_key
from .safe_keygen import analyze_rsa_params,
generate_safe_rsa_params
from .small_order import generate_small_order_demo,
recover_small_order_message
from .wiener import generate_wiener_vulnerable_key, wiener_attack


def _read_json(path: Path) -> dict[str, Any]:
    # JSON-файлы служат простым входным форматом для защиты и
    # отчета.
    return json.loads(path.read_text(encoding="utf-8"))


def _write_json(path: Path, data: dict[str, Any]) -> None:
    # Демо-команда сохраняет входы для всех атак в выбранный
    # каталог.
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, indent=2,
ensure_ascii=False), encoding="utf-8")


def _json_ready(value: Any) -> Any:
    if hasattr(value, "__dataclass_fields__"):
        return asdict(value)
    if isinstance(value, tuple):
        return [_json_ready(item) for item in value]
    if isinstance(value, list):
        return [_json_ready(item) for item in value]
    if isinstance(value, dict):
```

```

        return {key: _json_ready(item) for key, item in
value.items()}
    return value

def _cmd_common_modulus(args: argparse.Namespace) -> int:
    # Подкоманда запускает атаку общего модуля по n, e_b, d_b и
e_a.
    data = _read_json(Path(args.json))
    p, q, d_a = recover_from_known_private_exponent(
        int(data["n"]),
        int(data["e_b"]),
        int(data["d_b"]),
        int(data["e_a"]),
    )
    print(json.dumps({"p": p, "q": q, "d_a": d_a}, indent=2))
    return 0

def _cmd_wiener(args: argparse.Namespace) -> int:
    # Подкоманда проверяет, восстанавливается ли d атакой Винера.
    data = _read_json(Path(args.json))
    result = wiener_attack(int(data["n"]), int(data["e"]))
    print(json.dumps({"found": result is not None, "result":
result}, indent=2))
    return 0

def _cmd_broadcast(args: argparse.Namespace) -> int:
    # Подкоманда применяет широковещательную атаку к списку пар
(n_i, c_i).
    data = _read_json(Path(args.json))
    pairs = [(int(item["n"]), int(item["c"])) for item in
data["pairs"]]
    message = recover_broadcast_message(int(data["e"]), pairs)
    print(json.dumps({"message": message}, indent=2))
    return 0

def _cmd_small_order(args: argparse.Namespace) -> int:
    # Подкоманда выполняет бесключевое дешифрование через цикл
повторного
    # возведения в степень e.
    data = _read_json(Path(args.json))
    message, iterations = recover_small_order_message(
        int(data["n"]),

```

```

        int(data["e"]),
        int(data["c"]),
    )
    print(json.dumps({"message": message, "iterations":
iterations}, indent=2))
    return 0

def _cmd_safe_keygen(args: argparse.Namespace) -> int:
    # Подкоманда генерирует учебный RSA-ключ с базовыми проверками
    безопасности
    # и сохраняет закрытые параметры в artifacts.
    private = generate_safe_rsa_params(args.bits)
    output = Path(args.out)
    save_private_key(output, private)
    analysis = analyze_rsa_params(private)
    print(json.dumps(_json_ready({"private_key": private,
"analysis": analysis}), indent=2))
    return 0

def _build_common_modulus_demo() -> dict[str, Any]:
    # Демо для отчета: два открытых показателя используют один
    общий модуль.
    private_b, _ = generate_keypair(256, 65537)
    e_a = 17
    phi = (private_b.p - 1) * (private_b.q - 1)
    while True:
        try:
            private_a, _ = generate_keypair_from_existing_primes(private_b.p, private_b.q,
e_a)
            break
        except ValueError:
            e_a += 2
    recovered = recover_from_known_private_exponent(private_b.n,
private_b.e, private_b.d, e_a)
    return {
        "input": {"n": private_b.n, "e_b": private_b.e, "d_b":
private_b.d, "e_a": e_a},
        "expected": {"p": min(private_b.p, private_b.q), "q":
max(private_b.p, private_b.q), "d_a": private_a.d},
        "output": {"p": recovered[0], "q": recovered[1], "d_a":
recovered[2]},
        "phi": phi,
    }

```

```

def generate_keypair_from_existing_primes(p: int, q: int, e: int):
    from .rsa_core import generate_keypair_from_primes

    return generate_keypair_from_primes(p, q, e)

def _cmd_demo(args: argparse.Namespace) -> int:
    # Команда demo создает все JSON-входы и сводный файл
    # быстрой демонстрации на защите.
    output = Path(args.out)
    if output.exists():
        shutil.rmtree(output)
    output.mkdir(parents=True, exist_ok=True)

    common = _build_common_modulus_demo()
    _write_json(output / "common_modulus.json", common["input"])

    vulnerable_private, vulnerable_public =
generate_wiener_vulnerable_key(256)
    _write_json(
        output / "wiener.json",
        {"n": vulnerable_public.n, "e": vulnerable_public.e,
"d_real": vulnerable_private.d},
    )

    broadcast = generate_broadcast_demo(e=3, recipients=3,
bits=128)
    _write_json(
        output / "broadcast.json",
        {
            "e": broadcast["e"],
            "message_int": broadcast["message_int"],
            "pairs": [{"n": n, "c": c} for n, c in
broadcast["pairs"]],
        },
    )

    small = generate_small_order_demo()
    _write_json(output / "small_order.json", {"n": small["n"],
"e": small["e"], "c": small["c"]})

    summary = {
        "common_modulus": common,

```



```

        "wiener": {"recovered":
wiener_attack(vulnerable_public.n, vulnerable_public.e)},
        "broadcast": _json_ready(broadcast),
        "small_order": small,
    }
    _write_json(output / "demo_outputs.json", summary)
    print(f"Demo data written to {output}")
    return 0

def build_parser() -> argparse.ArgumentParser:
    # Парсер CLI фиксирует имена подкоманд и аргументы; их нельзя
    # менять без
    # обновления README, тестов и материалов отчета.
    parser = argparse.ArgumentParser(description="KMZI Lab2 RSA
attacks")
    subparsers = parser.add_subparsers(dest="command",
required=True)

    # common-modulus: восстановление p, q и d_a при общем n.
    common = subparsers.add_parser("common-modulus")
    common.add_argument("--json", required=True)
    common.set_defaults(func=_cmd_common_modulus)

    # wiener: атака на малый закрытый показатель d.
    wiener = subparsers.add_parser("wiener")
    wiener.add_argument("--json", required=True)
    wiener.set_defaults(func=_cmd_wiener)

    # broadcast: атака Хостада для одного сообщения без padding.
    broadcast = subparsers.add_parser("broadcast")
    broadcast.add_argument("--json", required=True)
    broadcast.set_defaults(func=_cmd_broadcast)

    # small-order: восстановление сообщения через короткий цикл e.
    small = subparsers.add_parser("small-order")
    small.add_argument("--json", required=True)
    small.set_defaults(func=_cmd_small_order)

    # safe-keygen: генерация параметров с проверками против учебных
    # слабостей.
    safe = subparsers.add_parser("safe-keygen")
    safe.add_argument("--bits", type=int, default=1024)
    safe.add_argument("--out", required=True)
    safe.set_defaults(func=_cmd_safe_keygen)

```

```
# demo: пакетная генерация входов для отчета и живой
демонстрации.
demo = subparsers.add_parser("demo")
demo.add_argument("--out", required=True)
demo.set_defaults(func=_cmd_demo)
return parser

def main(argv: list[str] | None = None) -> int:
    parser = build_parser()
    args = parser.parse_args(argv)
    return args.func(args)

if __name__ == "__main__":
    raise SystemExit(main())
```